Multi-Source Candidate Data Transformer

Technical Design Document - Prepared by Ishika (ishikajindaldav@gmail.com)

1. Pipeline / Step Breakdown

The transformer executes an asynchronous 5-stage pipeline designed for throughput, robustness, and auditability:

- **A. Ingest & Parse:** Accepts raw inputs. Features independent parsers for CSV sheets, ATS JSON records, GitHub API profile outputs, and unstructured text files (using regex heuristics).
- **B. Normalize & Clean:** Standardizes disparate fields: phones mapped to E.164; partial/full dates parsed to YYYY-MM; cities mapped to ISO-3166 alpha-2 country codes; raw skills normalized to standard dictionary spellings.
- **C. Deduplicate & Merge:** Matches candidate profiles by email keys (primary) or name+phone (secondary). Combines attributes based on source weight policies. Aggregates skills using a probabilistic OR union.
- **D. Project Output:** Executes custom schemas in real-time. Dynamically filters fields, paths, and arrays (e.g. wildcard selectors), maps new keys, override-normalizes, and handles missing fields.
- **E. Validate Schema:** Final stage validating the integrity of output datasets (both default canonical schema and custom config projections).

2. Canonical Output Schema & Standardized Formats

We define a strict, typed internal schema to prevent data pollution downstream:

- **phones:** E.164 standard (e.g. +12025550143). Cleans spaces, punctuation, and brackets, adding default country codes.
- **experience.start/end:** YYYY-MM format (e.g. 2018-01). Partial years default to -01. Present/Current jobs default to null.
- **location:** Structured object: { city, region, country }, mapping regions and resolving to ISO-3166-1 alpha-2 codes.
- **skills:** Canonicalized terms (e.g. JS -> JavaScript, node -> Node.js) with associated source arrays and confidence score.

3. Merge / Conflict-Resolution & Confidence Policy

To resolve duplicate/conflicting records when merging multiple files for a candidate, the system uses source weights:

Source Reliability Weights: ATS JSON (0.95) > Recruiter CSV (0.85) > GitHub Profile (0.75) > Recruiter Notes (0.60).

- Conflict Winner: For single-value fields (name, headline, location), the value from the source with the highest reliability score is selected. For lists, a union is performed.
- Probabilistic Skill Confidence: When skills are mentioned across multiple sources, we calculate a boosted confidence score using the complement probability formula: Confidence = 1 - " (1 - R_i). This mathematically models that independent multiple confirmations increase confidence.
- Audit Provenance: Every single field is linked to an array showing where the value was acquired (source file) and the ingestion mechanism (direct mapping or regex heuristic).

4. Runtime Custom-Output Projection & Validation

The projection layer uses a runtime configuration file to map the internal schema without code changes:

- Path Remapping: Supports extracting array indices (e.g. emails[0]), wildcards (e.g. skills[*].name), and deep paths.
- Normalization Overrides: Allows re-applying normalizers (E.164 or canonical) to fields at projection.

- Missing Fields Policy: Handled by "on_missing" configuration: null (sets value to null), omit (excludes field), or error (stops execution).

5. Edge Case Strategies & Exclusions

- **A. Missing Emails / IDs:** We fall back to matching Name + Phone combinations to avoid duplicating candidates who lack email addresses.
- **B. Timeline Overlaps:** When calculating years of experience, overlapping work periods are merged as single intervals to prevent inflating years of experience.
- **C. Garbage / Malformed Data:** Invalid values degrade gracefully into null. Fields with failed regex parsing are omitted or defaulted rather than causing crashes.
- **D. Time-Pressure Exclusion:** Under strict constraints, we deliberately omit full parsing of binary .docx/.pdf resumes in-house, opting instead to ingest pre-parsed plain text recruiter notes or public JSON representations.